

Proposed AI Integration Architecture

Project: HisabDo Website Enhancement

Department: AI/ML

Document: Proposed Integration Architecture

Prepared By: Muhammad Taha

Role: AI/ML Team Lead / AI Architect

Architecture: Modular, Secure, API-Driven, AI-Assisted Architecture

Status: Proposed Architecture

1. Executive Summary

This document defines the proposed integration architecture for integrating **HisabDo AI Copilot** with the existing HisabDo application.

The purpose of this architecture is to introduce AI capabilities into the existing HisabDo ecosystem without affecting the reliability, security, and business logic of the current application.

The proposed architecture separates the application into clearly defined layers:

React Frontend



Express Backend



FastAPI AI Service



AI Orchestrator



AI Modules



Verified Data / Knowledge Sources



Response Engine



User

The AI system will support future capabilities including:

- AI Chatbot
- FAQ Assistant
- AI Help and Support
- RAG-based Knowledge Search
- Multilingual AI
- Financial Intelligence
- Business Insights
- AI Recommendations
- User Engagement
- Voice AI
- Offline/Local AI

A core architectural principle is:

AI should understand and explain, while verified application services remain responsible for authentication, authorization, calculations, business logic, and financial data.

The Large Language Model must therefore **never be treated as the source of truth for financial information** and must not receive unrestricted database access.

2. Purpose of the Architecture

The purpose of this architecture is to define how AI capabilities will be integrated with the existing HisabDo platform.

This document specifically defines:

- Frontend integration points
- Backend/API integration points
- AI service boundaries
- Communication between React, Express, and FastAPI
- Chatbot request/response flow
- Knowledge Base access
- RAG integration
- Financial data access

- Authentication
 - Authorization
 - API security
 - Error handling
 - Fallback mechanisms
 - Scalability
 - Future AI integration
-

3. Architectural Goals

The proposed architecture aims to achieve the following goals:

3.1 Secure AI Integration

AI functionality must be integrated without exposing sensitive application or financial data unnecessarily.

3.2 Controlled Data Access

The AI must access information through controlled APIs and services rather than directly accessing databases.

3.3 Financial Accuracy

Financial values must come from verified HisabDo application data and business logic.

3.4 Modular AI Architecture

Individual AI capabilities should be independently developed and improved.

3.5 Scalable Design

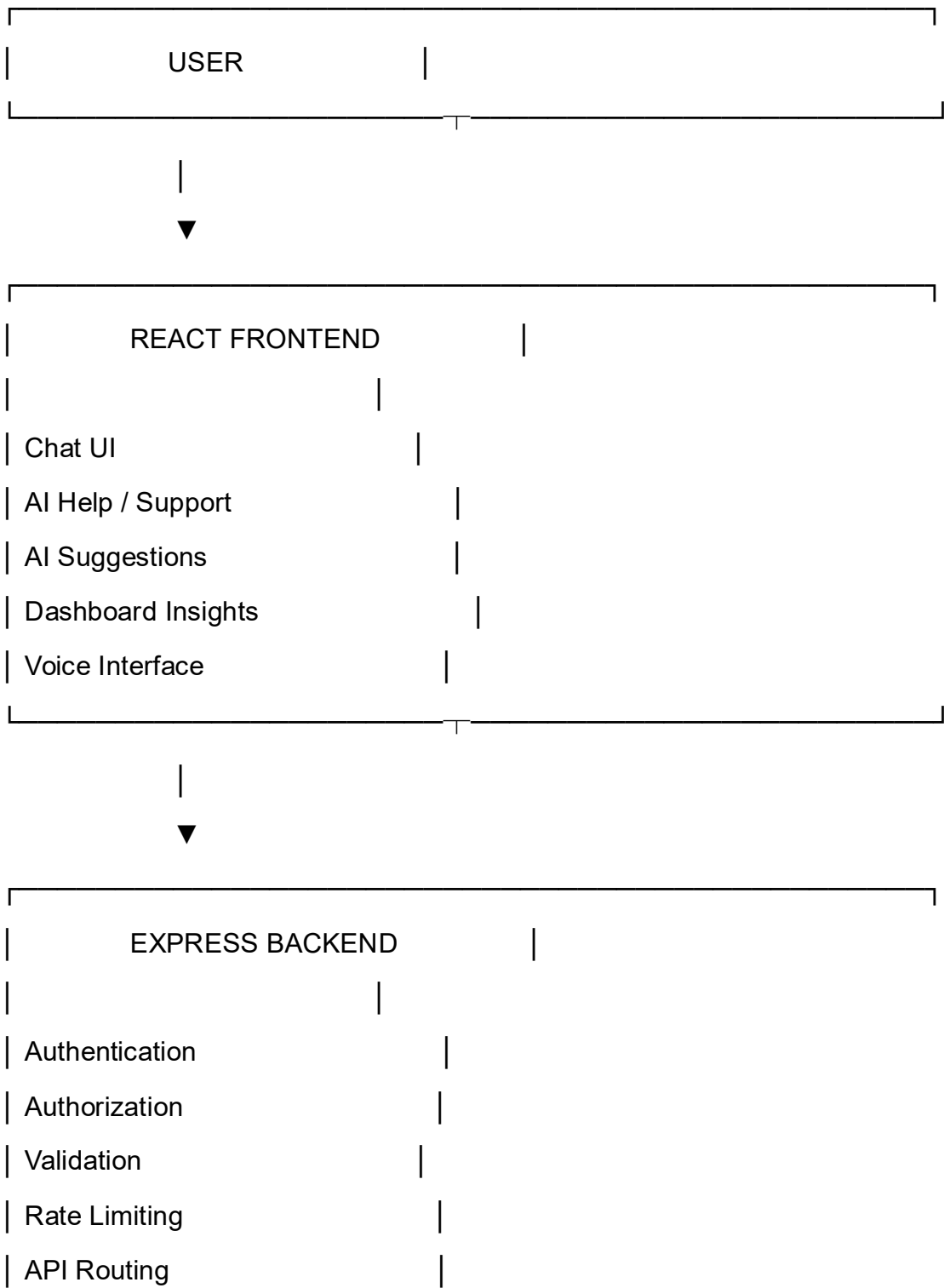
The architecture should support future growth in:

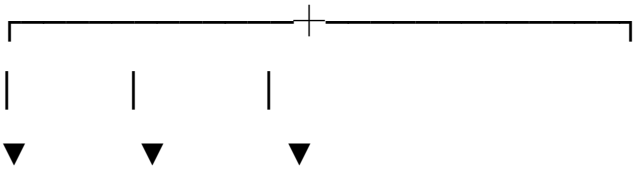
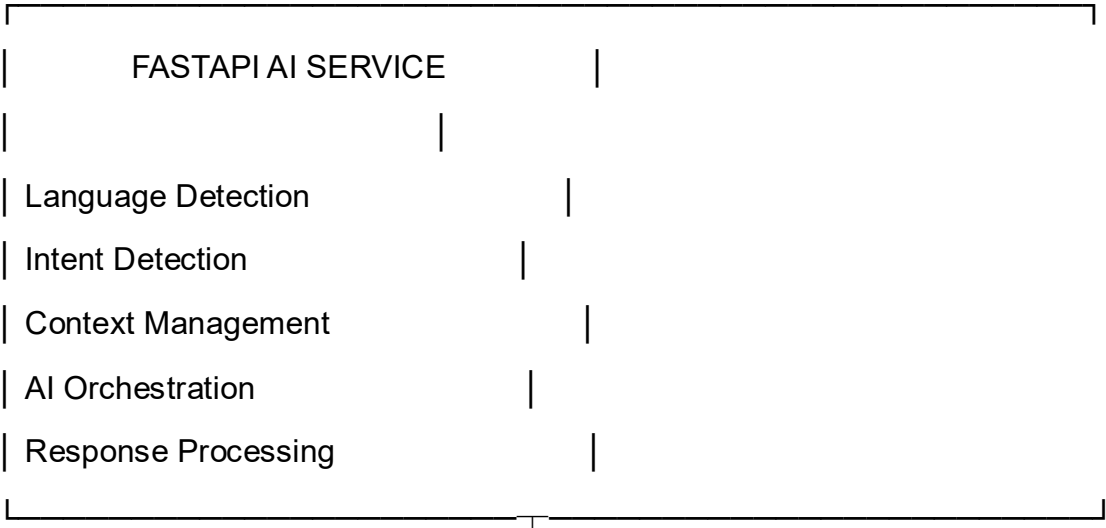
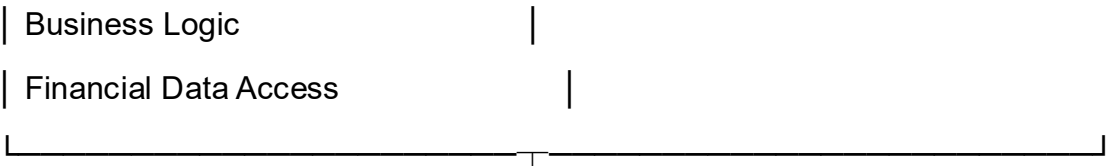
- Users
- AI requests
- Knowledge sources
- Languages
- AI modules
- Financial intelligence
- Recommendations

3.6 Maintainability

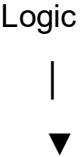
AI components should remain separated from the core application so that they can be updated without major changes to the existing system.

4. High-Level Architecture

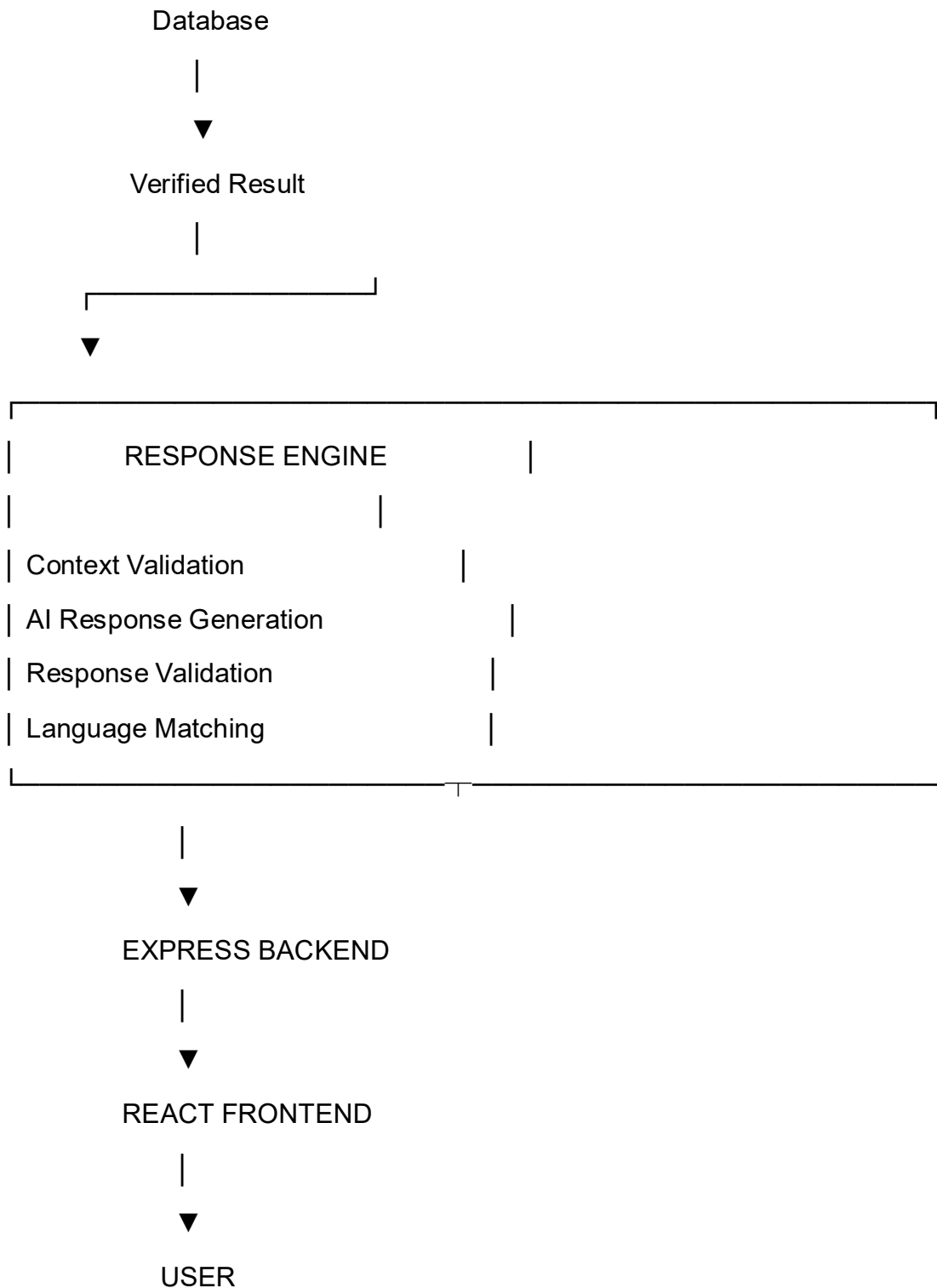




Knowledge Base Controlled API Analytics /
Vector Store / Business User Data



Logic



5. Architecture Layers

The proposed system consists of several logical layers.

Layer 1 — Presentation Layer

Responsible for user interaction.

Technology:

React

Components may include:

- AI Chat Interface
 - AI Help Widget
 - Smart Suggestions
 - Dashboard Insights
 - Recommendation Cards
 - Voice Interface
-

Layer 2 — Application/API Layer

Responsible for application security and business communication.

Technology:

Node.js

Express.js

Responsibilities:

- Authentication
 - Authorization
 - API validation
 - Rate limiting
 - Request routing
 - User context
 - Business logic
 - Financial API access
-

Layer 3 — AI Service Layer

Responsible for AI processing.

Technology:

Python

FastAPI

Responsibilities:

- Intent detection
 - Language detection
 - AI orchestration
 - RAG processing
 - Context management
 - AI response generation
 - Recommendation processing
-

Layer 4 — AI Module Layer

Contains specialized AI capabilities:

RAG Engine

FAQ Assistant

Financial AI

Insight Engine

Recommendation Engine

Multilingual Layer

Future Voice AI

Layer 5 — Data and Knowledge Layer

Contains:

- HisabDo database
 - Knowledge Base
 - FAQ documents
 - Product documentation
 - Vector database
 - Analytics data
-

6. Frontend Integration Architecture

The React frontend will provide the user-facing AI experience.

The frontend should communicate with the **Express backend**, not directly with the AI model or database.

Proposed Flow

React



Express API



AI Service

The React application may contain:

AI Chat Widget

Allows users to ask natural-language questions.

Examples:

How do I add a customer?

How can I export a report?

Ali ka kitna udhaar hai?

AI Help Widget

Provides contextual help based on the current feature.

Example:

User opens Backup page



AI Help



"Need help restoring your backup?"

Smart Suggestions

Examples:

- Review pending balances
 - Generate monthly report
 - Review expense trends
 - Explore a relevant HisabDo feature
-

Dashboard Insights

Future AI-generated insights may include:

- Expense trends
 - Outstanding balances
 - Customer activity
 - Transaction summaries
 - Follow-up suggestions
-

7. Backend Integration Architecture

The Express backend acts as the primary application gateway.

It is responsible for controlling access between the frontend and AI services.

Backend Responsibilities

Authentication

Authorization

Request Validation

Rate Limiting

Business Logic

Financial Data Access

AI Service Communication

Response Filtering

Error Handling

Logging

The backend must remain the authority for application and financial data.

8. AI Service Boundary

The FastAPI service is responsible for AI-specific processing.

It should not replace the existing application backend.

FastAPI Responsibilities

- Intent classification
- Language detection
- RAG orchestration
- Prompt management
- AI model communication
- Response generation
- Recommendation processing
- AI-specific validation

FastAPI Should Not Control

- User authentication
- User permissions
- Direct financial database access
- Critical financial calculations
- Direct financial transactions

These remain under the application backend.

9. Communication Architecture

The primary communication path is:

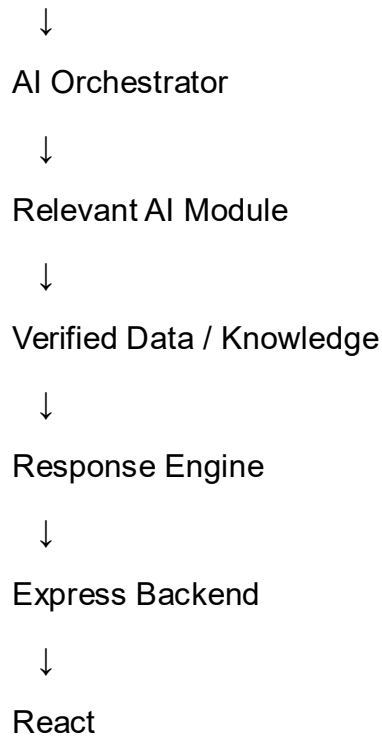
React



Express Backend



FastAPI AI Service



This separation prevents the frontend from directly interacting with internal AI infrastructure.

10. Chatbot Request Flow

When a user sends a chatbot message:

1. User enters message



2. React sends request



3. Express validates request



4. Authentication is verified



5. Request is sent to FastAPI



6. AI Orchestrator receives request



7. Language is detected



8. Intent is detected



9. Required AI module is selected



10. Required information is retrieved



11. AI generates response



12. Response is validated



13. Express returns response



14. React displays response

11. Intent Detection

The AI Orchestrator determines the purpose of the user's request.

Example:

Product Question

How do I create a customer?

Intent:

FEATURE_GUIDANCE

Module:

RAG / FAQ

Financial Question

Ali ka kitna udhaar hai?

Intent:

CUSTOMER_BALANCE_QUERY

Module:

Financial Data Service

Insight Request

Why did my expenses increase?

Intent:

BUSINESS_INSIGHT

Module:

Insight Engine

Recommendation Request

What should I check today?

Intent:

RECOMMENDATION_QUERY

Module:

Recommendation Engine

12. Knowledge Base Architecture

The Knowledge Base will contain verified HisabDo information.

Possible sources include:

Knowledge Base

|

|— FAQs

|— Feature Documentation

- └─ User Guides
- └─ Help Articles
- └─ Product Information
- └─ Website Content
- └─ Blog Content

The AI should use these sources when answering product-related questions.

13. Knowledge Base Access Flow

User Question



Intent Detection



Knowledge Query



Semantic Search



Relevant Content



Context Construction



LLM



Response

If relevant knowledge cannot be found, the AI should avoid confidently inventing product-specific information.

14. RAG Integration Architecture

RAG stands for **Retrieval-Augmented Generation**.

The RAG system retrieves relevant HisabDo content before generating a response.

Document Processing

HisabDo Documents



Cleaning



Chunking



Embedding Generation



Vector Store

Query Processing

User Question



Query Embedding



Similarity Search



Relevant Chunks



Prompt Context



LLM



Response

15. RAG Components

The RAG system may contain:

Document Loader

Loads knowledge sources.

Text Processor

Cleans and normalizes content.

Chunking Layer

Breaks documents into smaller searchable sections.

Embedding Model

Converts text into vector representations.

Vector Database

Stores embeddings and supports similarity search.

Initial prototype:

FAISS

Future scalable option:

Qdrant

Retrieval Layer

Finds relevant knowledge for the user's question.

16. Financial AI Integration

Financial AI requires stricter controls than normal chatbot functionality.

The LLM must not calculate or invent critical financial values.

Correct Architecture

User Question



AI Intent Detection



Entity Extraction



Controlled Financial Request



Express Backend



Authorization



Business Logic



Database



Verified Financial Result



FastAPI AI Service



Natural Language Explanation



User

17. Example Financial Query

User:

Ali ka kitna udhaar hai?

AI identifies:

Intent:

CUSTOMER_BALANCE_QUERY

Customer:

Ali

The AI then requests verified information through a controlled application API.

Example:

```
{  
  "customer": "Ali",  
  "balance": 25000,  
  "currency": "PKR"
```

}

The AI may then generate:

Ali ka current outstanding balance PKR 25,000 hai.

The AI is explaining the verified result rather than creating the financial value.

18. Direct Database Access Prevention

This is one of the most important security requirements.

Not Allowed

LLM

↓

Direct Database

Required

LLM

↓

Controlled AI Tool

↓

Backend API

↓

Authorization

↓

Business Logic

↓

Database

↓

Verified Result

The LLM should never have:

- Database credentials
- Raw database access
- Arbitrary SQL execution

- Unrestricted MongoDB queries
-

19. Authentication Architecture

AI requests involving user-specific data must be authenticated.

The backend should verify:

- User identity
- Session/token validity
- Account status
- Required permissions

The AI service should receive only the necessary user context.

Recommended Principle

Client-provided identity

≠

Trusted identity

The backend should derive trusted user identity from the authenticated request context.

20. Authorization

Authentication answers:

Who is the user?

Authorization answers:

What is the user allowed to access?

The AI system must respect the existing application's authorization rules.

Example:

User A

↓

User A's data only

The AI must not expose:

User B's customers

User B's transactions

User B's balances

21. Financial Data Isolation

Financial data must remain isolated by user/account/tenant according to the application's existing data model.

Before any financial information is returned:

User Request



Identity Verification



Authorization



Data Ownership Check



Business Logic



Verified Data

This prevents accidental cross-user data exposure.

22. API Security Requirements

AI APIs should implement appropriate security controls.

Required Controls

- Authentication
- Authorization
- Input validation
- Rate limiting
- Request size limits
- Secure service communication

- Error sanitization
 - Logging
 - Access control
 - Data minimization
-

23. Input Validation

AI endpoints should validate incoming requests.

Validation should cover:

- Required fields
- Message length
- Supported language values
- Conversation ID format
- Request size
- Malformed input

Example:

Invalid Request

↓

Validation Layer

↓

Reject Request

The AI service should not process malformed or clearly invalid requests.

24. Rate Limiting

AI operations can be computationally expensive.

Rate limiting should therefore be applied to prevent:

- Abuse
- Excessive requests
- Resource exhaustion
- Automated spam

Example:

User



AI Request



Rate Limit Check

|— Allowed → Continue

|— Exceeded → Reject / Retry Later

25. Response Validation

AI-generated responses should pass through a response validation layer before reaching the user.

Validation may check:

- Response structure
- Required fields
- Unsupported claims
- Sensitive information
- Expected response type
- Financial context

For financial responses, the system should ensure the response is based on the verified result returned by the application.

26. Anti-Hallucination Architecture

The system should distinguish between:

Verified Data

Database

Business Logic

Application APIs

and:

AI Knowledge

RAG

LLM

General Language Understanding

For financial questions:

Verified Data



AI Explanation

For product questions:

Verified Knowledge



AI Explanation

If verified information is unavailable, the system should communicate uncertainty rather than inventing an answer.

27. Error Handling

The system must handle failures at different layers.

Frontend Failure

Display a user-friendly message.

Backend Failure

Return a controlled API error.

AI Service Failure

Fallback to a predefined response or non-AI functionality.

RAG Failure

Use an appropriate fallback or inform the user that the requested information could not be retrieved.

Financial API Failure

Do not generate a guessed financial value.

28. Fallback Architecture

Recommended fallback flow:

Primary AI Response



Failure?



RAG / Knowledge Response



Failure?



Rule-Based Assistance



Failure?



Controlled Error Message

For financial information:

Verified Financial Data



Unavailable?



DO NOT GUESS



Inform User

29. Example Error Responses

AI Service Unavailable

The AI assistant is temporarily unavailable. Please try again later.

Knowledge Not Found

I couldn't find verified information about this topic.

Unknown Request

I couldn't fully understand your request. Please provide a little more detail.

Financial Data Unavailable

I couldn't retrieve the latest financial information right now. Please try again.

30. Service-to-Service Communication

Express and FastAPI should communicate through controlled internal APIs.

Example:

Express Backend



Internal AI API



FastAPI

The internal AI service should not be publicly exposed unnecessarily.

Where applicable, service-to-service authentication should be used.

31. Proposed API Structure

Chat

POST /api/ai/chat

Knowledge Search

POST /api/ai/knowledge/search

Financial Query

POST /api/ai/financial/query

Insights

GET /api/ai/insights

Recommendations

GET /api/ai/recommendations

Health

GET /api/ai/health

32. Standard Chat Request

```
{  
  "message": "How do I add a customer?",  
  "conversationId": "conversation_id",  
  "language": "en"  
}
```

The authenticated user identity should come from the backend authentication context rather than being trusted from the client payload.

33. Standard AI Response

```
{  
  "success": true,  
  "type": "knowledge_answer",  
  "message": "You can add a customer from the customer management section.",  
  "language": "en",  
  "source": {  
    "type": "knowledge_base"  
  }  
}
```

A financial response may additionally include structured verified data.

34. Conversation Context

The AI Copilot should support controlled conversation context.

Example:

User:

Ali ka balance batao.

AI:

Ali ka outstanding balance PKR 25,000 hai.

User:

Uski last transaction kab hui thi?

The system understands that:

"Uski" = Ali

Conversation context may include:

- Conversation ID
- Recent messages
- User language
- Current entity/context
- Session metadata

Sensitive information should not be retained unnecessarily.

35. Multilingual Integration

The AI service should support the multilingual architecture defined for HisabDo AI Copilot.

Potential languages include:

- English
- Urdu
- Roman Urdu
- Hindi
- Arabic

Flow:

User Input



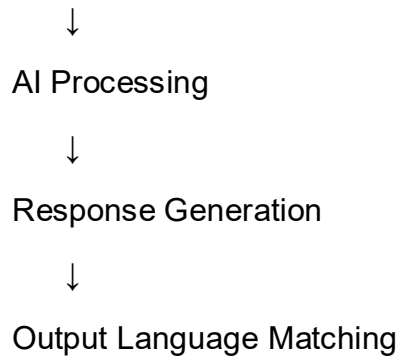
Language Detection



Input Normalization



Intent Detection



For example:

Roman Urdu Input



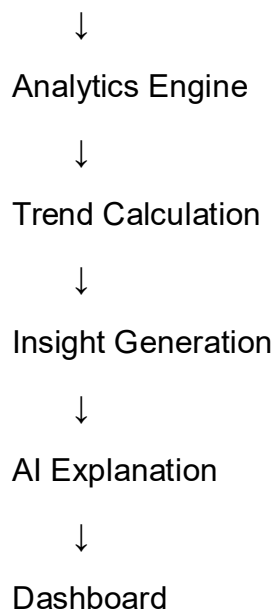
The system should also handle common variations in Roman Urdu spelling.

36. AI Insight Integration

Future dashboard integration can use the AI Insight Engine.

Example:

Transaction Data



Important calculations should be performed by deterministic application/analytics logic rather than relying on the LLM.

37. Recommendation Integration

The Recommendation Engine can provide:

- Smart feature suggestions
- Follow-up suggestions
- Relevant reports
- Customer follow-up recommendations
- Expense review suggestions
- Personalized feature discovery

Example:

User Activity



Rule / Analytics Engine



Recommendation



AI Explanation



React UI

38. AI User Engagement Integration

Future AI engagement features can include:

- Personalized AI suggestions
- Smart notifications
- Daily/weekly summaries
- AI-powered financial tips
- Context-aware recommendations
- Smart feature discovery
- AI follow-up suggestions
- Personalized dashboard insights

These features should be introduced gradually after the core AI assistant is stable.

39. Future Voice AI Integration

Voice AI can later be integrated as:

Voice Input



Speech-to-Text



AI Orchestrator



Knowledge / Financial / Insight Module



AI Response



Text-to-Speech



Voice Output

Voice processing should follow the same authentication and data-access rules as text-based AI.

40. Scalability Architecture

The AI service should be designed as a modular service rather than a tightly coupled component.

Initial architecture:

React



Express



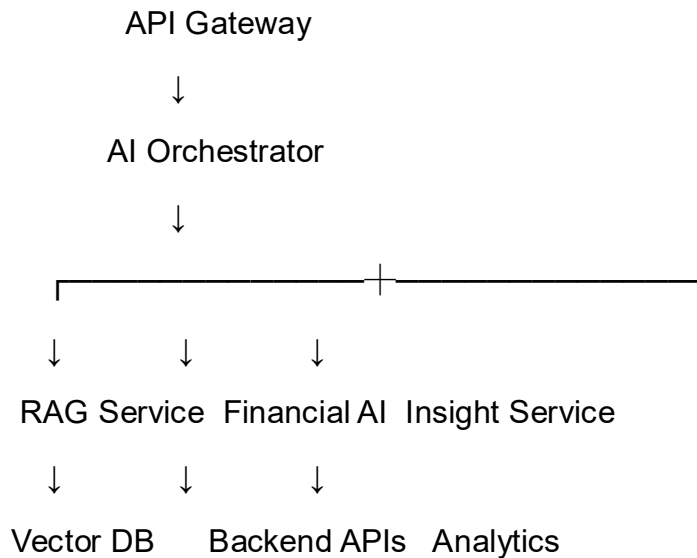
FastAPI



AI Modules

As demand increases, individual components can be scaled independently.

Potential future architecture:



41. Scalability Requirements

The architecture should support:

Horizontal Scaling

Multiple AI service instances can process requests.

Independent Module Scaling

High-demand AI modules can be scaled separately.

Vector Database Scaling

The knowledge retrieval system should support growth in documents and queries.

Model Flexibility

The architecture should allow different LLMs to be tested or replaced without changing the complete application.

Asynchronous Processing

Long-running operations may later use background jobs or queues.

42. Reliability Requirements

The system should provide:

- Health checks
- Timeout handling
- Retry policies where appropriate
- Service monitoring
- Structured logging
- Error tracking
- AI service availability monitoring

Example:

GET /api/ai/health

can be used to verify AI service availability.

43. Observability and Logging

The architecture should support controlled logging of:

- Request ID
- Endpoint
- Intent
- Processing status
- Response type
- Error information
- Processing time

Sensitive financial information should not be unnecessarily written to logs.

A unique request ID can help trace failures across services.

44. Performance Considerations

AI response time can be affected by:

- LLM inference
- Vector search
- Network communication
- Database queries

- Large prompts
- Multiple AI operations

Performance optimization should therefore include:

- Small relevant RAG context
 - Efficient vector search
 - Controlled prompt size
 - Request timeouts
 - Caching where appropriate
 - Lightweight models for suitable tasks
 - Async processing for long-running operations
-

45. Cost Optimization

The architecture should support cost-efficient AI operation.

Potential strategies include:

- Local LLMs for suitable workloads
- Ollama for development/local inference
- Lightweight models
- RAG instead of sending large documents to the LLM
- Caching repeated knowledge responses
- Rule-based processing for simple tasks
- Deterministic analytics for calculations

The AI system should not use an LLM for tasks that can be safely handled through normal application logic.

46. Technology Integration

The proposed architecture aligns with the current AI technology plan:

Component	Proposed Technology
Frontend	React
Backend	Node.js / Express.js

Component	Proposed Technology
AI Service	Python / FastAPI
Local LLM	Ollama
Embeddings	Sentence Transformers
Vector Store	FAISS
Future Vector DB	Qdrant
Application Database	Existing HisabDo Database
API Communication	REST APIs

The exact technologies can be finalized during implementation and benchmarking.

47. Development Environment

Initial AI development can use:

React

↓

Node.js / Express

↓

FastAPI

↓

Ollama

↓

Local LLM

↓

FAISS

This provides a practical environment for prototyping before production deployment.

48. Deployment Evolution

Prototype

Single Environment

+

Local AI

+

FAISS

Development

Frontend

+

Backend

+

FastAPI

+

Vector Store

+

LLM

Production

Frontend



API Layer



AI Gateway



Scalable AI Services



Vector Database



Verified Application APIs

49. Module Ownership

The integration architecture should follow clear ownership.

AI Architecture / Integration

Muhammad Taha — AI Lead / Architect

Responsibilities:

- Architecture
- Integration design
- Security architecture
- AI orchestration
- API boundaries
- Technical review
- Documentation
- Final architecture decisions

Chatbot

Chatbot Engineer

Responsibilities:

- Chat UI
- Chat API
- LLM integration
- Conversation handling

RAG

RAG Engineer

Responsibilities:

- Knowledge Base
- Chunking
- Embeddings
- Vector Search
- RAG pipeline

AI Research / Data

AI Research/Data Engineer

Responsibilities:

- AI use cases
 - Evaluation dataset
 - Multilingual testing
 - Recommendation research
 - AI testing
-

50. Integration Dependencies

The implementation should follow dependency order.

Existing Backend APIs



Authentication



AI Service



AI Orchestrator



Knowledge Base



RAG



Chatbot



Financial AI



Insights



Recommendations



Advanced AI

This prevents teams from implementing modules before their required infrastructure is available.

51. Implementation Priority

The recommended implementation order is:

Phase 1

AI Service Setup

+

Basic Chat API

+

Intent Detection

+

Basic FAQ

Phase 2

Knowledge Base

+

Embeddings

+

FAISS

+

RAG

Phase 3

Multilingual Support

+

Roman Urdu

+

Input Normalization

Phase 4

Financial Intent Detection

+

Controlled Financial APIs

+

Verified Financial Responses

Phase 5

Insights

+

Recommendations

+

User Engagement

Phase 6

Voice AI

+

Advanced Recommendations

+

Predictive Insights

+

Offline AI

52. Testing Strategy

Integration testing should be performed at multiple levels.

Unit Testing

Test:

- Intent detection
- RAG retrieval
- API validation
- Response formatting

Integration Testing

Test:

React



Express



FastAPI



AI Module

Security Testing

Test:

- Unauthorized requests
- Invalid tokens
- Cross-user data access
- Prompt injection attempts
- Excessive requests

Financial Testing

Test:

- Correct customer identification
- Correct balance retrieval
- Incorrect customer names
- Missing financial records
- Multiple customers with similar names

53. AI Evaluation

The AI system should be evaluated using:

Chatbot

- Intent accuracy
- Response relevance
- Response latency

RAG

- Retrieval accuracy
- Context relevance
- Unsupported response rate

Financial AI

- Data accuracy
- User isolation
- API correctness

Multilingual AI

- English
 - Urdu
 - Roman Urdu
 - Hindi
 - Arabic
-

54. Security Principles

The complete integration should follow these principles:

Principle 1

Never trust AI-generated financial values.

Principle 2

Never allow direct LLM database access.

Principle 3

Always enforce authorization before retrieving user data.

Principle 4

Use minimum necessary data.

Principle 5

Validate AI inputs and outputs.

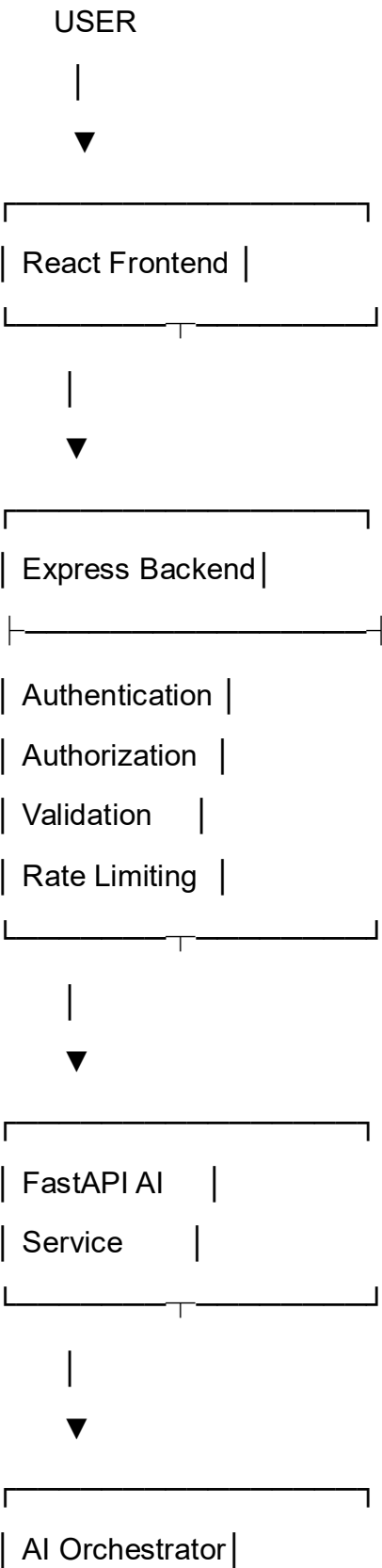
Principle 6

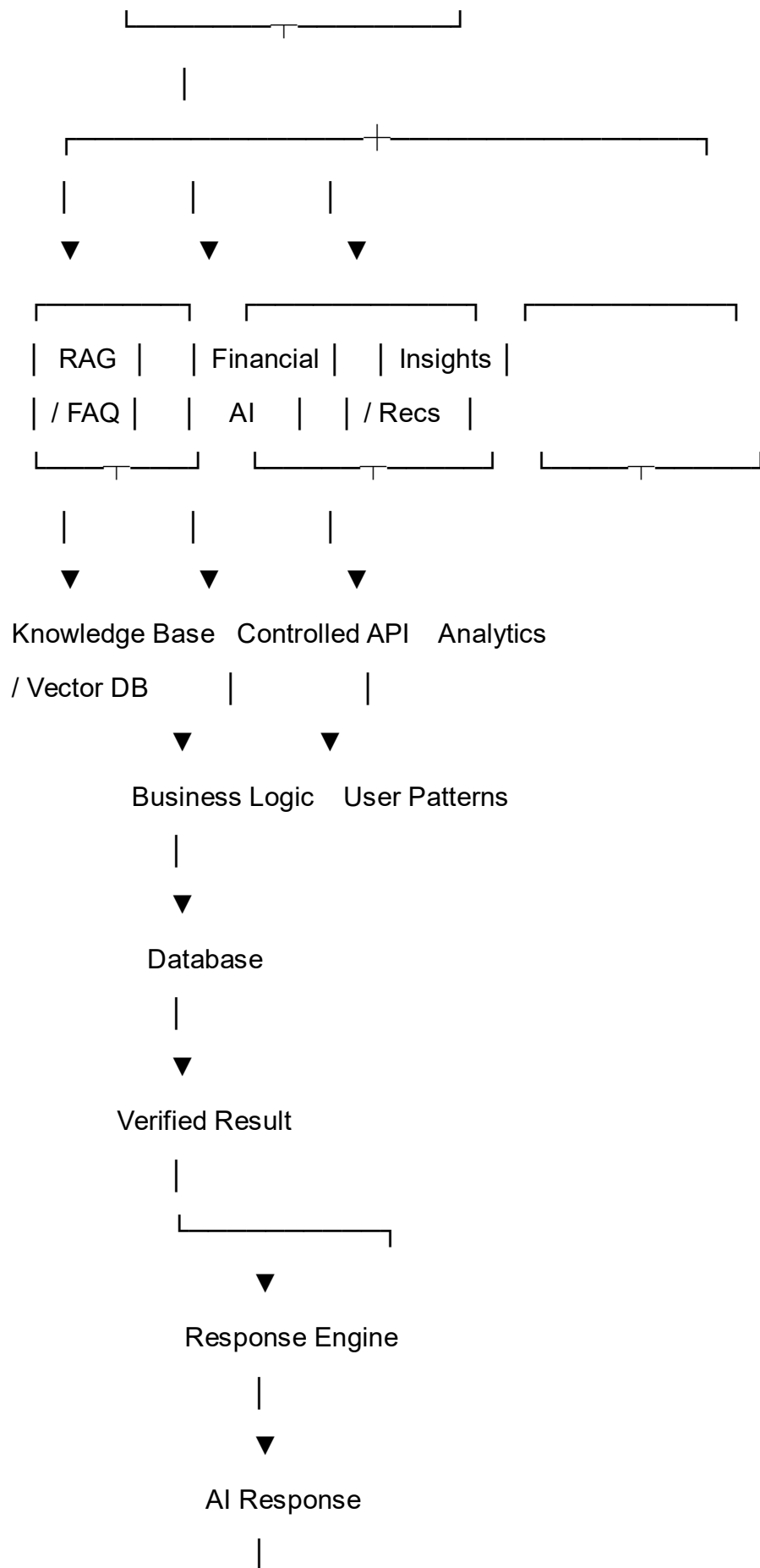
Maintain clear separation between AI and application business logic.

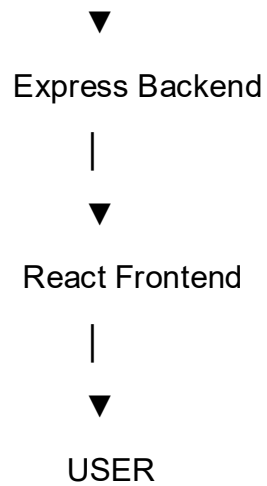
Principle 7

Fail safely when verified information is unavailable.

55. Complete End-to-End Flow







56. Future Architecture Vision

The proposed architecture provides a foundation for evolving HisabDo into a complete AI-powered financial platform.

Future capabilities may include:

AI Copilot

+

RAG

+

Multilingual AI

+

Financial Intelligence

+

Business Insights

+

Recommendations

+

User Engagement

+

Voice AI

+

Predictive Analytics

+

Offline AI

The architecture is intentionally modular so that these capabilities can be introduced incrementally.

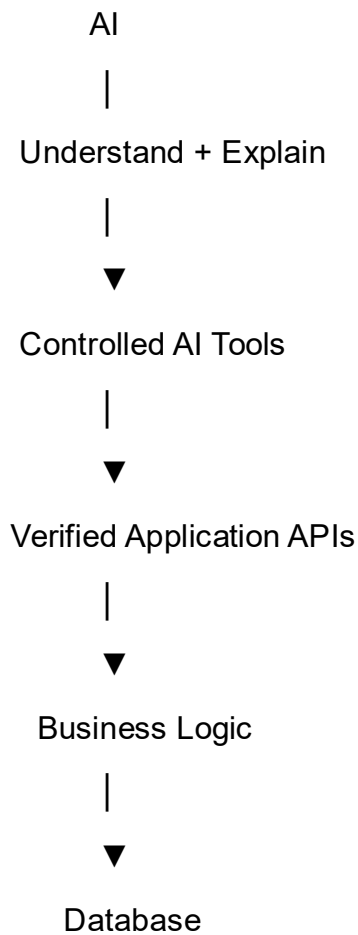
57. Key Architectural Decisions

The following decisions are proposed:

Area	Decision
Frontend AI Integration	React → Express
AI Backend	FastAPI
AI Orchestration	Dedicated AI Orchestrator
Knowledge Retrieval	RAG
Initial Vector Store	FAISS
Future Vector Store	Qdrant
Financial Data	Controlled Backend APIs
LLM Database Access	Not permitted
Authentication	Existing Backend Authentication
Authorization	Existing Backend Authorization
Financial Calculations	Application/Business Logic
AI Role	Understanding + Explanation
Data Source of Truth	HisabDo Application Data
Failure Handling	Controlled fallback
Scaling	Modular service architecture

58. Final Architecture Principle

The most important architectural principle of HisabDo AI Copilot is:



Therefore:

The LLM is not the source of truth.

The application remains responsible for:

- Authentication
- Authorization
- Financial calculations
- Business rules
- Database access
- Data ownership
- Critical actions

The AI layer is responsible for:

- Understanding natural language
- Detecting intent
- Retrieving knowledge

- Selecting appropriate AI modules
- Explaining verified results
- Providing recommendations and assistance

59. Final Proposed Flow

React Website



Express Backend



Authentication + Authorization



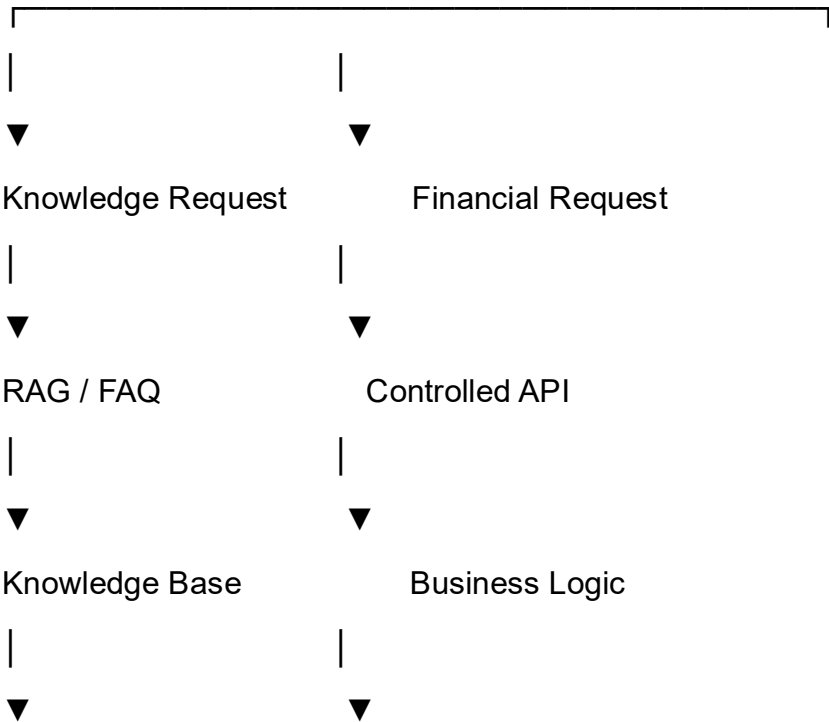
FastAPI AI Service

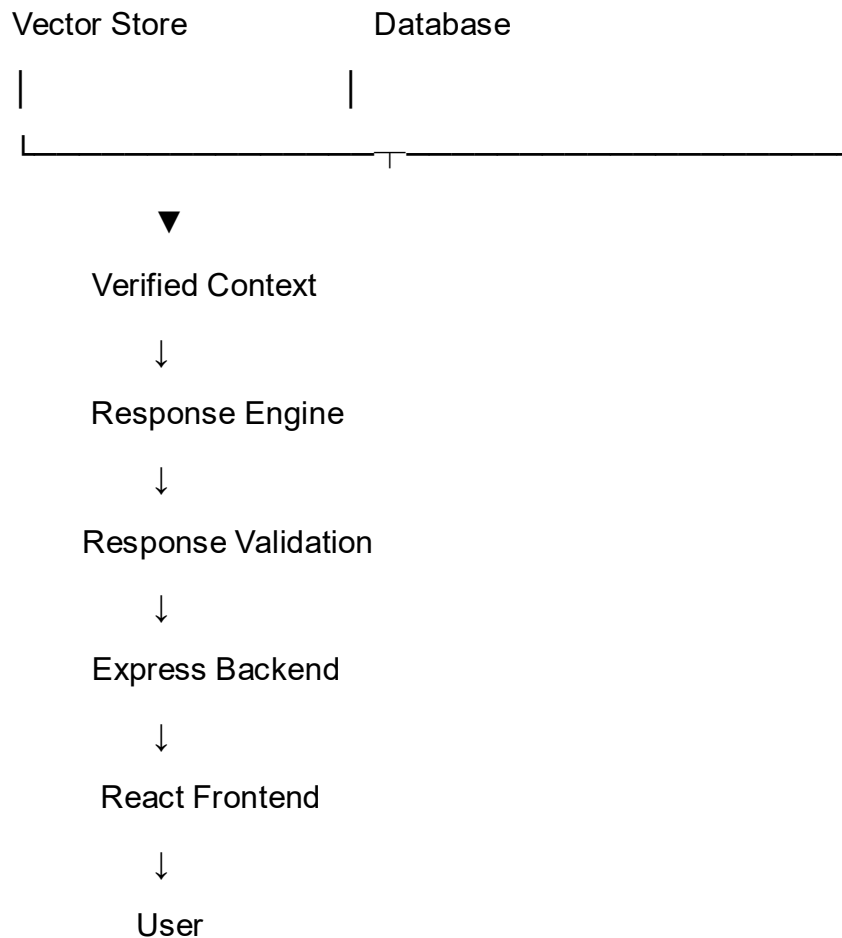


AI Orchestrator



Intent + Language Detection





60. Conclusion

The proposed integration architecture provides a secure, modular, scalable, and practical foundation for integrating **HisabDo AI Copilot** into the existing HisabDo application.

The architecture ensures that AI capabilities can be introduced without making the LLM responsible for critical application operations.

The system separates responsibilities clearly:

React

→ User Experience

Express

→ Security + Application Control

FastAPI

→ AI Processing

AI Orchestrator

→ Intelligent Routing

RAG

→ Verified Product Knowledge

Financial APIs

→ Verified Financial Data

Business Logic

→ Calculations + Rules

Database

→ Source of Truth

LLM

→ Understanding + Explanation

This approach allows HisabDo to start with a practical AI prototype and progressively expand toward:

AI Chatbot → RAG → Multilingual AI → Financial Intelligence → Insights → Recommendations → User Engagement → Voice AI → Advanced/Offline AI

while maintaining the core requirements of **security, privacy, accuracy, maintainability, scalability, and controlled AI behavior.**

Prepared By:

Muhammad Taha

AI/ML Team Lead / AI Architect

HisabDo AI/ML Team